AISC Practical Viva Guide

Algorithms Used & Key Code Explanations

Experiments 2–9 • Python implementations • Revision notes

A concise guide to aims, algorithm steps, important code elements, complexity, and viva points.

Experiment 2 — Water Jug using BFS

Aim: Find exactly 2 litres in Jug A (capacity 4 L) using Jug B (capacity 3 L).

Algorithm used

Breadth-First Search (BFS) explores states level by level using a FIFO queue.

Algorithm steps

1. Start at (0,0), with goal (2,0).
2. Add the start state and path to a queue.
3. Remove the oldest state; if it is the goal, return the path.
4. Generate states by filling, emptying, and pouring between jugs.
5. Add unseen states to the queue and mark them visited.
6. Repeat until the goal is found or the queue is empty.

Key code elements

Element	Explanation
CAP_A, CAP_B	Jug capacities: 4 litres and 3 litres.
next_states(state)	Generates possible fill, empty, and pour outcomes.
deque	Queue storing state and path.
visited	Prevents revisiting states.
popleft()	Processes the oldest queued state first.

Complexity (high-level)

Time: O(V + E); space: O(V), where V is states and E is transitions.

Viva points

• BFS uses a queue.

• A state (3,2) means A has 3 L and B has 2 L.

• BFS finds a shortest path when every operation has equal cost.

Implementation note: Goal is hard-coded as (2,0); use (0,2) if the required goal is 2 litres in Jug B.

Experiment 3 — Water Jug using A* Search

Aim: Solve the water jug problem using informed search.

Algorithm used

A* ranks states using f(n) = g(n) + h(n), where g is cost so far and h estimates remaining cost.

Algorithm steps

1. Start from (0,0) and add it to a priority queue.
2. Remove the state with the smallest priority.
3. Skip states already visited.
4. Stop when Jug A contains 2 litres.
5. Generate successors and add them with updated path cost and heuristic.
6. Repeat until goal or empty queue.

Key code elements

Element	Explanation
heuristic(state)	Estimates the difference between Jug A's current amount and 2 L.
pq	Min-priority queue.
g + 1	Cost after one more operation.
g + 1 + heuristic(nxt)	Calculates f = g + h.
path + [nxt]	Records the route.

Complexity (high-level)

General A* can take exponential time and memory; performance depends on the heuristic.

Viva points

• A* is an informed search algorithm.

• g is actual cost; h is estimated remaining cost.

• Evaluation function: f(n) = g(n) + h(n).

Implementation note: The heuristic is based on volume difference, not operation count, so it is not guaranteed admissible for this cost model.

Experiment 4 — Alpha-Beta Pruning

Aim: Find a minimax value while pruning branches that cannot affect the decision.

Algorithm used

Alpha-beta pruning optimizes minimax: the maximizing player seeks high values and the minimizing player seeks low values.

Algorithm steps

1. Start at root with alpha = -infinity and beta = infinity.
2. At depth 3, return the leaf value.
3. At maximizing nodes, choose the largest child and update alpha.
4. At minimizing nodes, choose the smallest child and update beta.
5. Prune when beta <= alpha.
6. Return the value to the root.

Key code elements

Element	Explanation
depth == 3	Leaf-node base case.
maximizing	Indicates max or min level.
alpha	Best bound for maximizer.
beta	Best bound for minimizer.
beta <= alpha	Pruning condition.

Complexity (high-level)

Worst-case time O(b^d); favorable ordering can improve pruning substantially.

Viva points

• Pruning does not change the minimax result.

• Alpha is the maximizer's bound; beta is the minimizer's bound.

• With the provided leaf values, expected result is 5.

Implementation note: Code assumes a complete binary tree with depth 3 and eight leaf values.

Experiment 5 — Genetic Algorithm

Aim: Demonstrate selection, crossover, and mutation on six 5-bit chromosomes using fitness $f(x)=x^2$.

Algorithm used

A Genetic Algorithm is a population-based optimization method inspired by biological evolution.

Algorithm steps

1. Enter six valid 5-bit chromosomes.
2. Convert each to decimal and calculate fitness x^2.
3. Sort by fitness and select the two best parents.
4. Use a crossover point to create two offspring.
5. Choose whether to mutate offspring 1, offspring 2, both, or neither.
6. Flip selected bits if mutation is chosen.
7. Combine offspring with remaining population and show the best chromosome.

Key code elements

Element	Explanation
validate(ch)	Checks five binary bits.
fitness(ch)	Converts binary to decimal and squares it.
population.sort(...)	Ranks chromosomes by fitness.
p1[:point] + p2[point:]	Single-point crossover.
mutate(ch,pos)	Flips the selected bit.
choice	Selects mutation option.

Complexity (high-level)

For this small fixed population, runtime is negligible; generally depends on population size, chromosome length, and generations.

Viva points

• Chromosome represents a candidate solution.

• Selection chooses parents; crossover combines them; mutation changes a gene.

• One generation does not guarantee the global optimum.

Implementation note: Mutation positions are entered by the user; they are not selected randomly.

Experiment 6 — Single-Layer Perceptron (AND)

Aim: Train a perceptron to reproduce the AND truth table.

Algorithm used

The perceptron learning rule updates weights and bias using target minus predicted output.

Algorithm steps

1. Initialize weights and bias to zero.
2. Calculate net = w1*x1 + w2*x2 + b.
3. Output 1 if net >= 0; otherwise output 0.
4. Calculate error = target - output.
5. Update weights by learning_rate × error × input; update bias by learning_rate × error.
6. Repeat epochs until total error is zero or the limit is reached.
7. Print predictions for all four inputs.

Key code elements

Element	Explanation
X and Y	AND input combinations and targets [0,0,0,1].
activation(net)	Binary step activation.
error	Difference between target and output.
weight updates	Adjust parameters using perceptron rule.
total_error == 0	Stops after correct classification of all samples.

Complexity (high-level)

Training time O(E × N × d), with E epochs, N examples, and d features.

Viva points

• Weights control input influence; bias shifts the threshold.

• An epoch is one full pass through training data.

• SLP can solve AND but not XOR.

Implementation note: Activation returns 1 when net equals zero, so training corrects the initial all-zero parameters.

Experiment 7 — MLP Backpropagation (XOR)

Aim: Train a small multi-layer perceptron on XOR inputs.

Algorithm used

MLP uses forward propagation to predict and backpropagation to calculate error signals and update weights.

Algorithm steps

1. Define XOR inputs and targets [0,1,1,0].
2. Initialize weights and biases.
3. Calculate hidden activations using sigmoid.
4. Calculate output activation.
5. Calculate output error signal.
6. Propagate error signals to hidden neurons.
7. Update weights and biases using learning rate.
8. Repeat for the specified epochs and display predictions.

Key code elements

Element	Explanation
sigmoid(x)	Maps weighted sum to a value between 0 and 1.
derivative(y)	Sigmoid derivative y(1-y), where y is activated output.
wih, who	Input-to-hidden and hidden-to-output weights.
error_out	Output-layer learning signal.
error_hidden	Hidden-layer learning signals.
epochs	Number of training passes.

Complexity (high-level)

Approximate training cost O(E × N × W), where W is number of weights.

Viva points

• MLP has one or more hidden layers.

• Backpropagation computes error gradients for weight updates.

• XOR is not linearly separable.

• Binary prediction uses threshold 0.5.

Implementation note: This compact example has no convergence check or loss reporting; perfect XOR predictions are not guaranteed.

Experiment 8 — K-Means Clustering

Aim: Group six student records using marks and attendance with K=2.

Algorithm used

K-Means alternates between assigning each point to its nearest centroid and recalculating centroids as cluster means.

Algorithm steps

1. Load records as [marks, attendance].
2. Initialize two centroids.
3. Calculate each point's Euclidean distance to both centroids.
4. Assign the point to the nearer centroid.
5. Recalculate each centroid as the mean of assigned points.
6. Stop when centroids stop changing or 100 iterations are reached.
7. Print clusters and centroids.

Key code elements

Element	Explanation
data	Student marks and attendance features.
distance(p,c)	Euclidean distance between point and centroid.
clusters	Points assigned to each group.
new_centroids	Mean coordinates for each cluster.
new_centroids == centroids	Checks exact stability.

Complexity (high-level)

Typical iteration costs O(N × K × d), for N points, K clusters, and d features.

Viva points

• K-Means is unsupervised.

• Centroid is the mean of points in a cluster.

• Expected groups: S1–S3 and S4–S6.

• Approximate centers: (82.33,89.67) and (37.67,52.33).

Implementation note: Exact float equality is acceptable for this small example but tolerance-based comparison is more robust generally.

Experiment 9 — Fuzzy Membership Functions

Aim: Calculate how strongly a temperature belongs to fuzzy set Warm.

Algorithm used

A triangular membership function maps a temperature to a value from 0 to 1 using a lower endpoint a, peak b, and upper endpoint c.

Algorithm steps

1. Set a=20, b=30, c=40.
2. Read temperature x.
3. Return 0 when x <= a or x >= c.
4. Return 1 when x equals b.
5. For a < x < b, use (x-a)/(b-a).
6. For b < x < c, use (c-x)/(c-b).
7. Print the membership value.

Key code elements

Element	Explanation
triangular_mf	Implements the piecewise triangular function.
x <= a or x >= c	Membership is zero at/outside endpoints.
x == b	Peak membership is 1.
(x-a)/(b-a)	Rising slope.
(c-x)/(c-b)	Falling slope.
float(input())	Accepts integer or decimal temperatures.

Complexity (high-level)

Each membership calculation is O(1) time and O(1) extra space.

Viva points

• Fuzzy membership can take any value from 0 to 1.

• The parameters are a, b, c.

• 25°C -> 0.5; 30°C -> 1.0; 35°C -> 0.5.

• Lotfi A. Zadeh introduced fuzzy set theory in 1965.

Implementation note: The code calculates membership only; the practical also discusses plotting with NumPy and Matplotlib.

Rapid Viva Revision

Exp.	Algorithm / concept	Must-remember point
2	BFS	Queue; level-by-level exploration; visited states.
3	A* Search	f(n) = g(n) + h(n); priority queue.
4	Alpha-Beta Pruning	Prune if beta <= alpha.
5	Genetic Algorithm	Selection → crossover → mutation.
6	Single-Layer Perceptron	Weighted sum, bias, step activation, learning rule.
7	MLP / Backpropagation	Forward pass, then backward error propagation.
8	K-Means	Nearest centroid, then recompute means.
9	Fuzzy Membership	Membership degree from 0 to 1.

Viva strategy: For each experiment, explain the aim, name the algorithm, describe its main steps, define key variables/functions, and state one advantage or limitation.